



Security Audit Report

13/10/2025

Arbitrary Message Bridge

Content

Introduction	3
Disclaimer	3
Scope	4
Executive Summary.....	5
Conclusions	6
Vulnerabilities	7
List of vulnerabilities	7
Vulnerability details	7
Risk of Assets Becoming Stuck	8
Missing Message Size Validation	10
Arbitrary Execution Management Risks	12
Implementation Asymmetries.....	13
Missing Nonce Return on New Messages	15
Improper Storage Context	16
Lack of Inputs Validation	17
GAS Optimizations	18
Lack of Checks-Effects-Interactions Pattern	19
Missing Null Checks in Validators	20
[Out of Scope] Misplaced Storage Gap	21
Annexes.....	22
Methodology	22
Manual Analysis.....	22
Automatic Analysis.....	22
Vulnerabilities Severity.....	23

Introduction

The **Arbitrary Data Message Bridge** is a cross-chain communication system that enables secure message passing between N3 blockchain and EVM-compatible networks. The bridge allows users to: send arbitrary data, execute smart contract calls and retrieve execution results across different blockchain networks. The system implements a validator-based consensus mechanism equivalent to the one used by the Native/Token bridge, with the same access-control roles and similar configurations.



As solicited by **AxLabs** and as part of the vulnerability review and management process, Red4Sec has been requested to perform a security code audit in order to evaluate the security of the **Arbitrary Message Bridge** project.

The report includes specifics retrieved from the audit for all the existing vulnerabilities of Arbitrary Message Bridge. The performed analysis shows that the smart contract contains a high vulnerability.

Disclaimer

This document only represents the results of the code audit conducted by Red4Sec Cybersecurity and should not be used in any way to make investment decisions or as investment advice on a project.

Likewise, the report should not be considered either "endorsement" nor "disapproval" of the guarantee of the correct business model of the analyzed project, nor as guarantee on the operation or viability of the implemented financial product.

Red4Sec makes full effort and applies every resource available for each audit, however it does not warrant the function, nor the safety of the project and it cannot be deemed a sufficient assessment of the code's utility and safety, bug-free status, or any other declarations of the project. Additionally, Red4Sec makes no security assessments or judgments about the underlying business strategy, or the individuals involved in the project.

Blockchain technology and cryptographic assets come with their own new risks and challenges, where the ecosystem, platform, its programming language, and other software related to said technology can have vulnerabilities that could lead to exploits. As a result, the audit cannot guarantee the explicit security of the audited projects.

The audit reports can be used to improve the code quality of smart contracts, to help limit the vectors of attack and to lower the high level of risks associated with utilizing new and continually changing technologies such as cryptographic tokens and blockchain, but they are unable to detect any future security concerns with the related technologies.

Scope

Red4Sec Cybersecurity has made a thorough audit of the **Arbitrary Message Bridge** security level against attacks, identifying possible errors in the design, configuration, or programming; therefore, guaranteeing the availability, integrity and confidentiality of the project and the possible assets treated and stored.

The scope of this evaluation includes the following items provided by **AxLabs**:

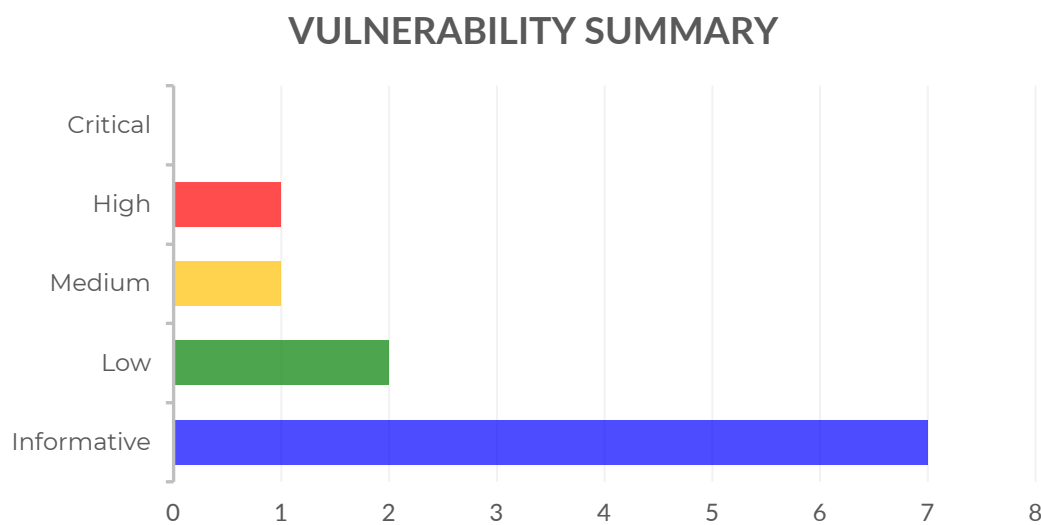
- EVM Contracts
 - <https://github.com/bane-labs/bridge-evm-contracts>
 - Commit: f87e5ca5cb1212e085f3d06860002e9f5c87c6ea
 - Smart Contracts:
 - contracts/library/AMBStorage.sol
 - contracts/library/AMBTypes.sol
 - contracts/library/MessageBridgeLib.sol
 - contracts/messageBridge/ExecutionManager.sol
 - contracts/messageBridge/MessageBridge.sol
 - contracts/messageBridge/interfaces/IExecutionManager.sol
 - contracts/messageBridge/interfaces/IMessageBridge.sol
- N3 Contracts
 - <https://github.com/bane-labs/bridge-neo-contracts>
 - Commit - 07f7d0662e34c9ff1f08ad3c84add14a81b739fb
 - Smart Contracts:
 - src/main/java/network/bane/lib/MessageBridgeLib.java
 - src/main/java/network/bane/message/DeploymentData.java
 - src/main/java/network/bane/message/MessageBridgeContract.java
 - src/main/java/network/bane/message/MessageBridgeContractHelper.java
 - src/main/java/network/bane/message/MessageBridgeImpl.java
 - src/main/java/network/bane/message/StorageConstants.java
 - src/main/java/network/bane/messageexecution/ExecutionManagerContract.java
 - src/main/java/network/bane/structs/message/ExecutableState.java
 - src/main/java/network/bane/structs/message/MessageBridge.java
 - src/main/java/network/bane/structs/message/N3Message.java
 - src/main/java/network/bane/structs/message/N3MessageEnvelope.java
 - src/main/java/network/bane/structs/message/N3MethodCall.java

Executive Summary

The security audit against **Arbitrary Message Bridge** has been conducted between the following dates: **04/08/2025** and **20/08/2025**.

Once the analysis of the technical aspects has been completed, the performed analysis shows that the audited source code contains non-critical vulnerabilities.

During the analysis, a total of **11 vulnerabilities** were detected, these vulnerabilities have been classified by the following level of risks, defined in Vulnerabilities Severity Annex.



Conclusions

Based on the audit conducted, it is concluded that the **Arbitrary Message Bridge** satisfactorily achieves the purpose for which it was developed. The contracts are secure and no critical vulnerabilities have been found beyond the "Risk of Assets Becoming Stuck", which poses a high risk due to its potential impact on the locking of funds.

The overall impression of the code quality and organization is optimal, and the project follows good development practices and is well documented. The successful functioning of the project depends on external entities that are beyond the scope of the current audit, such as the bridge nodes. It is therefore recommended to perform periodic audits on these external entities to ensure their security.

The most significant vulnerability identified is the "Risk of Assets Becoming Stuck" which holds a high risk. It is recommended to implement mechanisms that allow the recovery of assets in case they become stuck due to unforeseen situations.

The "Missing Message Size Validation" with medium risk could allow malicious actors to submit overly large messages that cause unexpected behavior. It is advised to include validation of message sizes to prevent potential misuse.

The "Arbitrary Execution Management Risks" and "Implementation Asymmetries" have been classified as low risk. Nonetheless, they should be addressed to further enhance the security of the smart contract.

The remaining vulnerabilities such as "Missing Nonce Return on New Messages", "Improper Storage Context", "Lack of Inputs Validation", "GAS Optimizations", "Lack of Checks-Effects-Interactions Pattern", and "Missing Null Checks in Validators" have been categorized as informative. While these do not pose immediate security risks, it is recommended to address them to improve the quality and robustness of the code.

Though the project is quite secure, there is always room for enhancements in terms of code quality and security practices. It is recommended to prioritize the issues mentioned above and implement improvements as needed to boost overall security and mitigate potential risks.

The AxLabs team has successfully addressed all the issues identified during the audit. Fixes have been verified, and the previously reported vulnerabilities have been resolved satisfactorily. The final state of the code reflects a robust and secure implementation aligned with best practices.

Vulnerabilities

In this section, you can find a detailed analysis of the vulnerabilities encountered upon the security audit.

List of vulnerabilities

Below, we have gathered a complete list of the vulnerabilities detected by Red4Sec, presented, and summarized in a way that can be used for risk management and mitigation.

Table of vulnerabilities			
ID	Vulnerability	Risk	State
AMB-01	Risk of Assets Becoming Stuck	High	Fixed
AMB-02	Missing Message Size Validation	Medium	Fixed
AMB-03	Arbitrary Execution Management Risks	Low	Fixed
AMB-04	Implementation Asymmetries	Low	Fixed
AMB-05	Missing Nonce Return on New Messages	Informative	Fixed
AMB-06	Improper Storage Context	Informative	Fixed
AMB-07	Lack of Inputs Validation	Informative	Fixed
AMB-08	GAS Optimizations	Informative	Fixed
AMB-09	Lack of Checks-Effects-Interactions Pattern	Informative	Assumed
AMB-10	Missing Null Checks in Validators	Informative	Fixed
AMB-11	[Out of Scope] Misplaced Storage Gap	Informative	Assumed

Vulnerability details

In this section, we provide the details of each of the detected vulnerabilities indicating the following aspects:

- Category
- Active
- Risk
- Description
- Recommendations

Risk of Assets Becoming Stuck

Identifier	Category	Risk	State
AMB-01	Business Logic	High	Fixed

A risk of asset lock-up has been identified in the `ExecutionManager` contract when silent failure of external executions is allowed. The `executeMessage` function, marked as `payable`, enables the contract to receive native tokens during cross-chain message execution. However, if the external execution fails and the `allowFailure` flag is set to `true`, the function does not revert, resulting in the permanent retention of the sent funds within the contract without any recovery mechanism.

```
function executeMessage(
    uint256, // nonce
    bytes calldata rawMessage
)
    external
    payable
    override
    onlyRole(BRIDGE_ROLE)
    returns (AMBTypes.Result memory result)
{
    // If rawMessage contains data that doesn't match the structure of the Call struct
    // the transaction will fail with a decoding error
    AMBTypes.Call memory call = abi.decode(rawMessage, (AMBTypes.Call));

    (bool success, bytes memory returnData) = _executeCall(call.target, call.value, call.callData);
    if (!success && !call.allowFailure) revert ExecutionFailed(returnData);

    result = AMBTypes.Result({success: success, returnData: returnData});
}
```

This design may lead to scenarios where legitimate funds sent as part of a failed cross-chain execution become locked, with no available mechanism for withdrawal or reuse. Since the `ExecutionManager` contract lacks any fund extraction functionality (e.g., `withdraw`, `sweep`) and does not appear to include internal logic for reutilizing these assets, any failure under `allowFailure = true` results in an effective loss of the involved native tokens.

Such behavior can lead to user or integrator confusion, unintended fund losses, and degrades the reliability of the bridge. Additionally, it introduces technical debt by accumulating irretrievable balances in a component that, by design, should not act as a custodian of funds.

Recommendations

- Include a controlled function that allows recovery of accumulated native assets in the contract in the event of failed executions.
- Alternatively, the logic can be redesigned so that if `call.allowFailure` is `true` and the execution fails, the funds are immediately refunded to the bridge contract or to the originator of the request.
- Reject messages where `allowFailure == true` and `value > 0`, ensuring that value is only attached when execution success is mandatory.

Source Code References

- `contracts/messageBridge/ExecutionManager.sol#L21-L39`
- `contracts/messageBridge/MessageBridge.sol#L373`

Fixes Review

This issue has been addressed in the following pull request:

- <https://github.com/bane-labs/bridge-evm-contracts/pull/298>

Missing Message Size Validation

Identifier	Category	Risk	State
AMB-02	Data Validation	Medium	Fixed

The absence of a maximum message size validation has been identified in the bridge implementation on the N3 blockchain (NeoVM). This omission allows messages of arbitrary length to be processed by the contract, exposing the system to potential resource exhaustion and denial-of-service attacks.

Unlike the EVM implementation, where the message size is explicitly checked against a configured limit (`config.maxMessageSize`), the corresponding NeoVM implementation lacks such validation.

```
function _sendMessageWithMetadata(bytes memory _message, bytes memory _encodedMetadata) private {
    AMBStorage.MessageConfig memory config = AMBStorage.getConfig();

    // Check message size against max allowed size
    if (_message.length > config.maxMessageSize) revert MessageTooLarge(config.maxMessageSize, _message.length);

    // Process the fee for message sending
    address from = msg.sender;
    _processBridgeFee(from, msg.value, config.fee);
}
```

EVM side

The `sendMessage` method, which initiates the message sending process from N3, accepts a `rawMsg` parameter of type `ByteString` and passes it without restriction to the `serializeAndUpdateEvmMessageState` function.

```
public static int sendMessage(ByteString rawMsg, Hash160 feeSponsor, int maxFee) {
    MessageBridgeImpl.payMessageSendingFee(feeSponsor, maxFee);

    int timestamp = Runtime.getTime();
    Hash160 callingScriptHash = Runtime.getCallingScriptHash();

    N3Message.N3MetadataStoreOnly metadata = new N3Message.N3MetadataStoreOnly(timestamp, callingScriptHash);
    return serializeAndUpdateEvmMessageState(metadata, rawMsg);
}
```

N3 side

Although a configuration variable `messageBridge.config.maxBytesForSending` exists and can be set via the `setMaxBytesForSending` method, it is not referenced in any part of the message validation flow, indicating a possible inconsistency or incomplete implementation.

The absence of a size constraint allows attackers to send excessively large messages, potentially leading to disproportionate gas consumption, contract congestion, or failures in receiving systems that enforce bounded message sizes. Furthermore, the lack of symmetry between the EVM and NeoVM constraints introduces an inconsistency in the bridge logic that may undermine the system's interoperability and reliability.

Recommendations

- Introduce explicit message size validation in the `sendMessage` and `sendExecutableMessage` methods to ensure that the message length does not exceed the value defined in `messageBridge.config.maxBytesForSending`.
- Ensure that the configured size limit is consistent with the constraint enforced in the EVM implementation (`config.maxMessageSize`) to guarantee symmetry and compatibility across both ends of the bridge.

Source Code References

- `src/main/java/network/bane/message/MessageBridgeImpl.java#L241`
- `contracts/messageBridge/MessageBridge.sol#L242`
- `src/main/java/network/bane/message/MessageBridgeImpl.java#L74-L79`

Fixes Review

This issue has been addressed in the following pull request:

- <https://github.com/bane-labs/bridge-neo-contracts/pull/221>

Arbitrary Execution Management Risks

Identifier	Category	Risk	State
AMB-03	Design Weaknesses	Low	Fixed

The bridge contract design enables users to send cross-chain messages that trigger arbitrary contract execution on the destination network. While this inherent risk has been partially mitigated through the use of a dedicated `ExecutionManager` contract, significant attack vectors remain due to delegated execution within a privileged context.

In the EVM environment, it is critical to ensure that `ExecutionManager` holds no administrative privileges in the `BridgeManagement`. This could allow an attacker to craft malicious executions that impersonate the `ExecutionManager` contract in interactions with external contracts.

In the N3 (NeoVM) implementation, the risks are even more pronounced. While this is not currently a vulnerability, it is important to note that, under Neo's execution and permission model, if tokens were ever allowed to be held by the `MessageBridge` or `ExecutionManager` contracts, an inadequately restricted `ExecutionManager` could potentially perform actions such as governance voting, native asset transfers (e.g., GAS or NEO), or arbitrary contract calls on behalf of the bridge. Additionally, improper use of `WitnessScope` in signatures can amplify these risks, allowing actions with the full capabilities of the invoker, posing a risk to the entity initiating the execution.

While these vectors do not constitute a concrete technical vulnerability in the current codebase, they represent architectural risks that must be explicitly acknowledged and proactively managed through strict controls and ongoing review, particularly as new functionalities or external contracts are integrated into the bridge system.

Recommendations

- Ensure that the `ExecutionManager` contract does not hold assets or possess privileges in any other contract deployed as part of the bridge system.
- Restrict the `WitnessScope` used during execution to prevent calls from being made with the privileges of the bridge or other users.

References

- <https://developers.neo.org/docs/n3/foundation/Transactions#signature-scope>

Source Code References

- `contracts/messageBridge/ExecutionManager.sol#L35`
- `src/main/java/network/bane/messageexecution/ExecutionManagerContract.java#L98`

Fixes Review

This issue has been partially addressed in the following pull request:

- <https://github.com/bane-labs/bridge-neo-contracts/pull/209>

Implementation Asymmetries

Identifier	Category	Risk	State
AMB-04	Design Weaknesses	Low	Fixed

Minor functional discrepancies have been identified between the EVM and N3 (NeoVM) implementations of the system, which may compromise behavioral consistency and integration experience. While certain execution environment differences may justify specific design choices, divergent functionalities can hinder the development of cross-chain applications that rely on predictable behavior across both ends of the bridge.

- Firstly, the EVM implementation includes the `allowFailure` parameter in the `executeMessage` method, enabling arbitrary calls to fail silently without reverting the transaction. In contrast, the N3 implementation lacks this option, causing failed executions to revert the entire operation. This can result in inconsistent behavior for identical messages processed across networks.
- Secondly, execution state management methods such as `getExecutingNonce`, `setExecutingNonce`, and `unsetExecutingNonce` exist solely in the N3 implementation, with no EVM counterpart. These methods allow explicit tracking of in-progress executions, but their absence in EVM introduces asymmetry in execution flow visibility and control.
- Thirdly, the `executeMessage` method in the N3 `ExecutionManager` enforces an additional security constraint by disallowing calls directed to itself, reducing the attack surface. The EVM version imposes no such restriction, allowing calls to any address, which introduces a significant difference in exposure.
- Lastly, as outlined in the issue "*Missing Message Size Validation*" N3 lacks message size validation, unlike EVM, further contributing to inconsistent validation logic between the two architectures.

While these differences do not constitute immediate technical vulnerabilities, they present operational and architectural risks that should be reviewed and addressed by the development team.

Recommendations

- Unify the behavior of the `allowFailure` parameter across both implementations, or explicitly document the divergence.
- Consider implementing EVM equivalents of `getExecutingNonce`, `setExecutingNonce`, and `unsetExecutingNonce` to achieve functional parity and enhance execution state traceability.
- Evaluate incorporating an optional execution target restriction in EVM, mirroring the protection enforced in the N3 implementation.

Source Code References

- `contracts/messageBridge/ExecutionManager.sol#L36-L37`
- `src/main/java/network/bane/messageexecution/ExecutionManagerContract.java#L75-L85`
- `src/main/java/network/bane/messageexecution/ExecutionManagerContract.java#L95`
- `contracts/messageBridge/MessageBridge.sol#L242`

Fixes Review

This issue has been addressed in the following pull requests:

- <https://github.com/bane-labs/bridge-neo-contracts/pull/221>
- <https://github.com/bane-labs/bridge-evm-contracts/pull/299>

Missing Nonce Return on New Messages

Identifier	Category	Risk	State
AMB-05	Design Weaknesses	Informative	Fixed

The public methods responsible for sending messages in the EVM bridge implementation (`sendMessage` and `sendExecutableMessage`) do not return the nonce associated with the newly generated message. As the nonce serves as a unique and incremental identifier for each message, its immediate availability is critical for tracking message status on the destination network or during processing.

In the current design, the nonce is computed within the internal `_sendMessageWithMetadata` function but is not returned by the public methods that invoke it. This differs from the NeoVM implementation, where the private `updateEvmMessageState` method returns the nonce, and it is subsequently exposed by the public methods, allowing users and external systems to immediately access the identifier of the sent message.

The lack of nonce return in EVM forces integrators to rely on emitted events (`MessageSent`) or perform additional state queries to retrieve the message identifier, introducing unnecessary complexity and increasing the risk of synchronization issues or traceability loss.

Recommendations

- Update the internal `_sendMessageWithMetadata` function to return the computed nonce, enabling its propagation through the contract logic.
- Modify the public `sendMessage` and `sendExecutableMessage` functions to return the generated nonce at the time of message submission.
- Align behavior between the EVM and NeoVM contracts regarding message identifier exposure to enhance developer experience and facilitate interoperability.

Source Code References

- `contracts/messageBridge/MessageBridge.sol#L238-L263`
- `contracts/messageBridge/MessageBridge.sol#L157`
- `contracts/messageBridge/MessageBridge.sol#L180`
- `src/main/java/network/bane/message/MessageBridgeImpl.java#L325`

Fixes Review

This issue has been addressed in the following pull request:

- <https://github.com/bane-labs/bridge-evm-contracts/pull/300>

Improper Storage Context

Identifier	Category	Risk	State
AMB-06	Design Weaknesses	Informative	Fixed

The N3 implementation contains multiple storage queries that do not adhere to the Principle of Least Privilege (PoLP). Several read-only functions access storage using a full-privilege context instead of a read-only context.

The N3 platform differentiates between storage contexts, with `getReadOnlyContext()` being the appropriate mechanism for safely accessing data without the ability to modify state.

Using the read-only context ensures that any attempt to write, whether accidental or due to a logic error, is rejected at runtime. Employing the standard (read-write) context in functions intended solely for reading unnecessarily exposes the contract to unintended state changes through future logic bugs, unsafe refactoring, or vulnerabilities introduced by subsequent code modifications.

Recommendations

- Replace the default storage context with `getReadOnlyContext()` in all read-only functions.
- Conduct a comprehensive code review to identify and correct any read access that uses a write-enabled context unnecessarily.

References

- https://en.wikipedia.org/wiki/Principle_of_least_privilege

Source Code References

- `src/main/java/network/bane/message/MessageBridgeImpl.java#L176`
- `src/main/java/network/bane/message/MessageBridgeImpl.java#L189`
- `src/main/java/network/bane/message/MessageBridgeImpl.java#L285`

Fixes Review

This issue has been addressed in the following pull request:

- <https://github.com/bane-labs/bridge-neo-contracts/pull/222>

Lack of Inputs Validation

Identifier	Category	Risk	State
AMB-07	Data Validation	Informative	Fixed

Certain methods in the bridge contracts lack explicit validation of input parameters. This omission can result in unexpected behavior, logical errors, or even execution failures.

Below are some of the most relevant cases identified during the audit:

Integer range

Some of the input integer values used are not validated against allowed ranges (no minimum or maximum checks when setting numeric values).

Source Code References

- `src/main/java/network/bane/message/MessageBridgeContract.java#L427`
- `src/main/java/network/bane/message/MessageBridgeContract.java#L438`
- `src/main/java/network/bane/message/MessageBridgeContract.java#L449`
- `src/main/java/network/bane/message/MessageBridgeContract.java#L460`

Zero address verification

In some methods it may be convenient to check that the value is not `IsZero`, leaving the verification as: `hash.IsValid && !hash.IsZero`.

Source Code References

- `src/main/java/network/bane/message/DeploymentData.java#L14-L15`

Fixes Review

This issue has been addressed in the following pull request:

- <https://github.com/bane-labs/bridge-neo-contracts/pull/219>

GAS Optimizations

Identifier	Category	Risk	State
AMB-08	Codebase Quality	Informative	Fixed

Software optimization is the process of modifying a software system to make an aspect of it work more efficiently or use less resources. This premise must be applied to smart contracts as well, so that they execute faster or in order to save GAS.

On EVM blockchain, GAS is an execution fee which is used to compensate miners for the computational resources required to power smart contracts. If the network usage is increasing, so will the value of GAS optimization.

Reuse Storage Pointer

Gas consumption can be reduced by reusing `AMBStorage.get()` calls that retrieve the storage structure for read or write operations. For example, in the `storeMessage` method, instead of repeatedly calling `AMBStorage.get()` to retrieve different values, the pointer can be stored in a local variable and reused throughout the function.

Source Code References

- `contracts/messageBridge/MessageBridge.sol#L302`
- `contracts/messageBridge/MessageBridge.sol#L307`
- `contracts/messageBridge/MessageBridge.sol#L369`

Variable Order Declaration

Gas can be saved in cases where execution is aborted early by deferring the declaration of `MessageBridge messageBridge = getMessageBridge()` until after all required validations have passed and immediately before its first use.

```
static void setSendingFee(int newFee) {  
    MessageBridge messageBridge = getMessageBridge();  
    if (newFee < 0) abort("Sending fee must be nonnegative");  
    messageBridge.config.sendingFee = newFee;  
    storeMessageBridge(messageBridge);  
}
```

Source Code References

- `src/main/java/network/bane/message/MessageBridgeImpl.java#L68`
- `src/main/java/network/bane/message/MessageBridgeImpl.java#L75`
- `src/main/java/network/bane/message/MessageBridgeImpl.java#L82`
- `src/main/java/network/bane/message/MessageBridgeImpl.java#L89`
- `src/main/java/network/bane/message/MessageBridgeImpl.java#L96`

Fixes Review

This issue has been partially addressed in the following pull request:

- <https://github.com/bane-labs/bridge-neo-contracts/pull/217>

Lack of Checks-Effects-Interactions Pattern

Identifier	Category	Risk	State
AMB-09	Timing and State	Informative	Assumed

A deviation from the recommended **Checks** → **Effects** → **Interactions** (CEI) pattern has been identified in the internal function `_sendMessageWithMetadata`, due to its invocation of `_processBridgeFee`, which handles bridge fee processing, in the EVM implementation. Although the function performs proper input validation and includes defensive measures such as rejecting refunds to contracts, it executes an external call before updating local state, violating the ideal CEI order.

The CEI pattern is a widely adopted best practice in smart contract development as it significantly reduces the risk of reentrancy vulnerabilities. In this specific case, the risk is mitigated by explicitly preventing ETH transfers to contracts using `_isContract`, and since Externally Owned Accounts (EOAs) do not execute code upon receiving ETH, the practical impact remains low.

Nonetheless, deviating from the CEI pattern introduces potential fragility, especially if future changes relax current restrictions or add new execution paths. Adhering strictly to CEI improves auditability, reduces the risk of regressions, and enhances code clarity and maintainability.

Recommendations

- Reorder operations within `_sendMessageWithMetadata` and `_processBridgeFee` so that state changes occur before any external calls.
- Alternatively, apply the `nonReentrant` modifier to prevent reentrancy if reordering is not feasible.

References

- <https://docs.soliditylang.org/en/latest/security-considerations.html#use-the-checks-effects-interactions-pattern>
- https://fravoll.github.io/solidity-patterns/checks_effects_interactions.html

Source Code References

- `contracts/messageBridge/MessageBridge.sol#L246`
- `contracts/messageBridge/MessageBridge.sol#L493`

Fixes Review

This issue has been addressed in the following pull request:

- <https://github.com/bane-labs/bridge-evm-contracts/pull/301>

Missing Null Checks in Validators

Identifier	Category	Risk	State
AMB-10	Codebase Quality	Informative	Fixed

Internal validation methods such as `isValid` fail to check whether the received object is `null` before accessing its internal fields or properties.

While these methods correctly validate the internal structure of the object, they lack a preliminary null check on the parameter itself, weakening defensive robustness. This design assumes that callers will always provide non-null inputs, an assumption that may break, particularly when inputs originate from dynamic data or external messages (as is common in bridge operations), increasing the risk of unexpected runtime failures in production.

A properly implemented example of the method would be the following:

```
public static boolean isValid(N3MessageEnvelope message) {  
    return message != null &&           // <- Avoid null errors  
        message.nonce != null &&  
        message.message != null &&  
        N3Message.isValid(message.message);  
}
```

Recommendations

- Update all `isValid` methods to include an explicit null check (`if (obj == null) return false;`) before accessing any fields of the object.

Source Code References

- `src/main/java/network/bane/structs/message/N3MessageEnvelope.java#L16`
- `src/main/java/network/bane/structs/message/N3Message.java#L29`
- `src/main/java/network/bane/structs/message/N3Message.java#L79`
- `src/main/java/network/bane/messageexecution/ExecutionManagerContract.java#L131`
- `src/main/java/network/bane/message/DeploymentData.java#L13`

Fixes Review

This issue has been addressed in the following pull request:

- <https://github.com/bane-labs/bridge-neo-contracts/pull/218>

[Out of Scope] Misplaced Storage Gap

Identifier	Category	Risk	State
AMB-11	Design Weaknesses	Informative	Assumed

A deviation from best practices in the design of upgradeable Solidity contracts has been identified in the `BridgeManagementStorageV1` contract. The `_gap0` variable, intended as a reserved storage gap for future expansion, has been defined at the beginning of the contract, before the actively used state variables. This placement is inconsistent with the recommended pattern for upgradeable contracts.

In proxy-based upgradeable contract architectures (e.g., OpenZeppelin's model), it is standard practice to define a storage gap at the end of the contract. This approach reserves space in the storage layout for future variables, ensuring that additions do not interfere with the existing layout and minimizing the risk of storage collisions. Placing the gap at the beginning introduces unnecessary complexity in future layout planning, as new variables cannot be added sequentially without overwriting the reserved space, defeating the primary purpose of the storage gap.

While this issue does not present an immediate security or functional risk, it may hinder future upgrades, increase the likelihood of migration errors, and potentially lead to data loss or state corruption in production environments.

Recommendations

- Relocate the definition of `uint256[100] private _gap0;` to the end of the `BridgeManagementStorageV1` contract, after all currently used state variables.

References

- <https://docs.openzeppelin.com/upgrades-plugins/writing-upgradeable#storage-gaps>

Source Code References

- `contracts/management/BridgeManagementStorageV1.sol#L11`

Annexes

Methodology

A code audit is a thorough examination of the source code of a project with the objective of identifying errors, discovering security breaches, or contraventions of programming standards. It is an essential component to the defense in programming, which seeks to minimize errors prior to the deployment of the product.

Red4Sec adopts a set of cybersecurity tools and best security practices to audit the source code of the smart contract by conducting a search for vulnerabilities and flaws.

The audit team performs an analysis on the functionality of the code, a manual audit, and automated verifications, considering the following crucial features of the code:

- The implementation conforms to protocol standards and adheres to best coding practices.
- The code is secure against common and uncommon vectors of attack.
- The logic of the contract complies with the specifications and intentions of the client.
- The business logic and the interactions with similar industry protocols do not contain errors or lead to dangerous situations to the integrity of the system.

In order to standardize the evaluation, the audit is executed by industry experts, in accordance with the following procedures:

Manual Analysis

- Manual review of the code, line-by-line, to discover errors or unexpected conditions.
- Assess the overall structure, complexity, and quality of the project.
- Search for issues based on the SWC Registry and known attacks.
- Review known vulnerabilities in the third-party libraries used.
- Analysis of the business logic and algorithms of the protocol to identify potential risk exposures.
- Manual testing to verify the operation, optimization, and stability of the code.

Automatic Analysis

- Scan the source code with static and dynamic security tools to search for known vulnerabilities.
- Manual verification of all the issues found by the tools and analyzes their impact.
- Perform unit tests and verify the coverage.

Vulnerabilities Severity

Red4Sec determines the severity of vulnerabilities found in risk levels according to the impact level defined by CVSSv3 (Common Vulnerability Scoring System) by the National Institute of Standards and Technology (NIST), classifying the risk of vulnerabilities on the following scale:

Severity	Description
Critical	Vulnerabilities that possess the highest impact over the systems, services and/or sensitive information. The existence of these vulnerabilities is dangerous and should be fixed as soon as possible.
High	Vulnerabilities that could severely compromise the service or the information it manages even if the vulnerability requires expertise to be exploited.
Medium	Vulnerabilities that on their own can have a limited impact and/or that combined with other vulnerabilities could have a greater impact.
Low	These vulnerabilities do not suppose a real risk for the systems. Also includes vulnerabilities which are extremely hard to exploit or whose impact on the service is low.
Informative	It covers various characteristics, information or behaviours that can be considered as inappropriate, without being considered as vulnerabilities by themselves.



Invest in Security, invest in your future